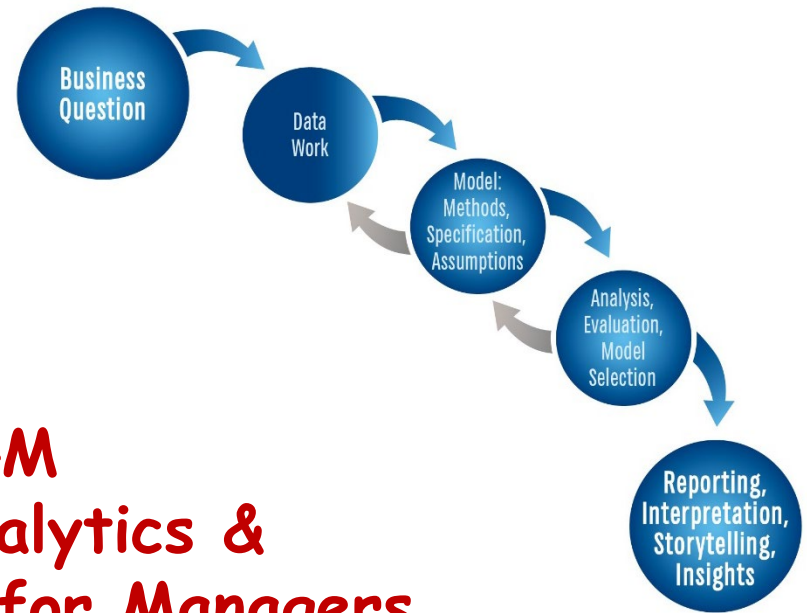




PAML4M Predictive Analytics & Machine Learning for Managers Neural Networks

Use PAML4M_11R_DeepLearning&NeuralNets.Qmd

Free online book: <http://neuralnetworksanddeeplearning.com>

Prof. J. Alberto Espinosa
Last updated 8/1/2026



KOGOD SCHOOL of BUSINESS
AMERICAN UNIVERSITY • WASHINGTON, DC

What we Know: Regression

What is linear regression? A number of **independent variables** predicting an **outcome variable**. The linear regression model predicts an outcome using a linear combination of the predictors, like a weighted sum, with each predictor **weighted** by its respective regression **coefficient**.

What is a non-linear regression? It is similar to a linear regression, but some variables are transformed (e.g., square, cube) to achieve a better fit of the regression line with the data.

Quantitative predictions → regression models aiming to predict a **quantitative outcome** (e.g., sales, profits, number of customers, etc.)

Classification predictions → regression models aiming to find the correct **classification of an outcome** into either: (1) one of two categories (i.e., binomial, e.g., approve/reject, stop vehicle/continue); or (2) more than two categories (i.e., multinomial, e.g., green light/amber/red; stop/slow down/continue)

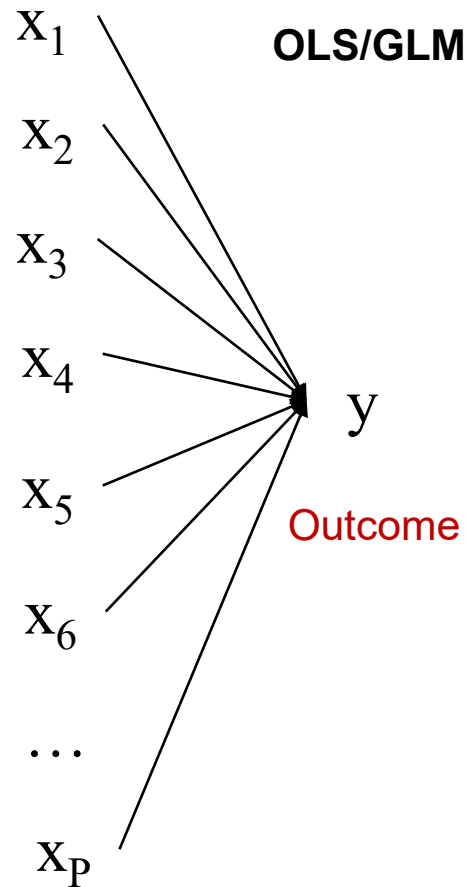
What we Know: Dimension Reduction

What are principal components regression (PCR) and partial least squared regression (PLSR)?

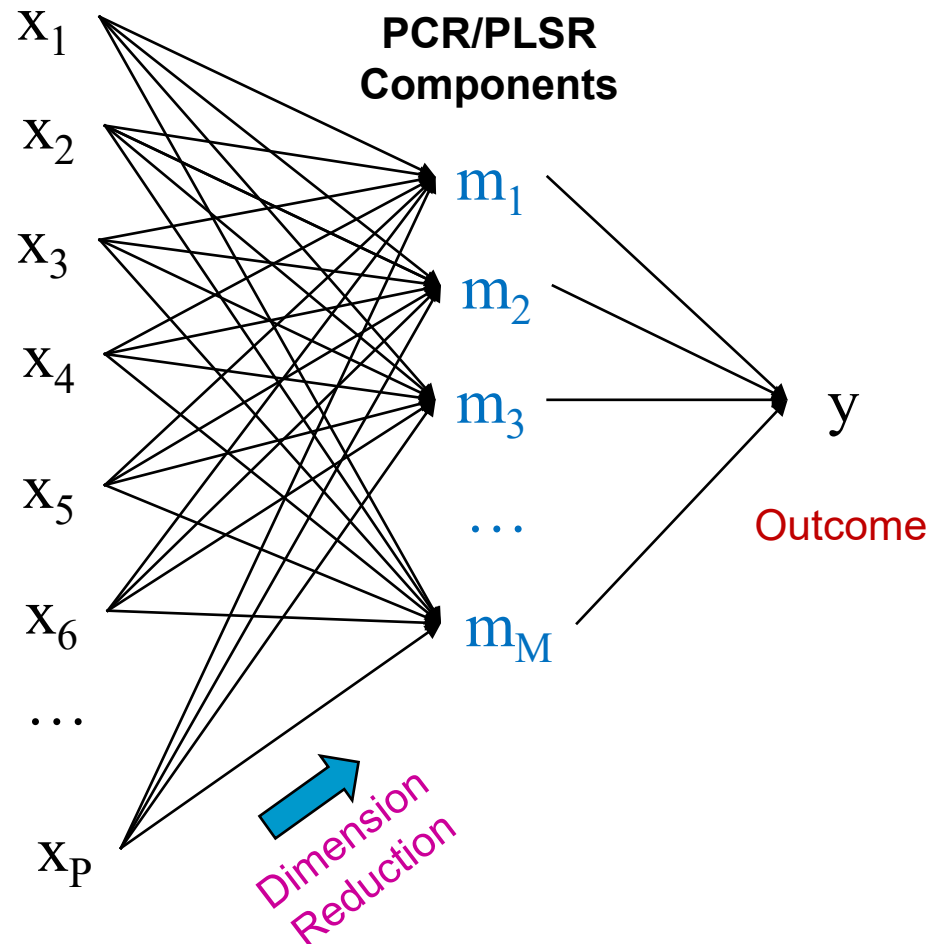
These are methods in which the input variables predict an outcome variable in stages:

- 1st → The input variables (i.e., predictors) estimate an intermediate artificial layer of variables called **Principal Components (PCs)**,
- 2nd → We then use these **PCs** to estimate the outcome variable.
- We employ **PCR** or **PLSR** when there is multicollinearity in the model with the goal of **reducing** the **dimensionality** of a model from **P** predictors to **$M \ll P$** components.
- It is like having **M** different **regressions**, with each component as an outcome variable in the first stage, which can then be used as predictors for the final outcome.
- **Neural networks** share some similarity with this concept.

Quantitative Regression & PCR/PLS

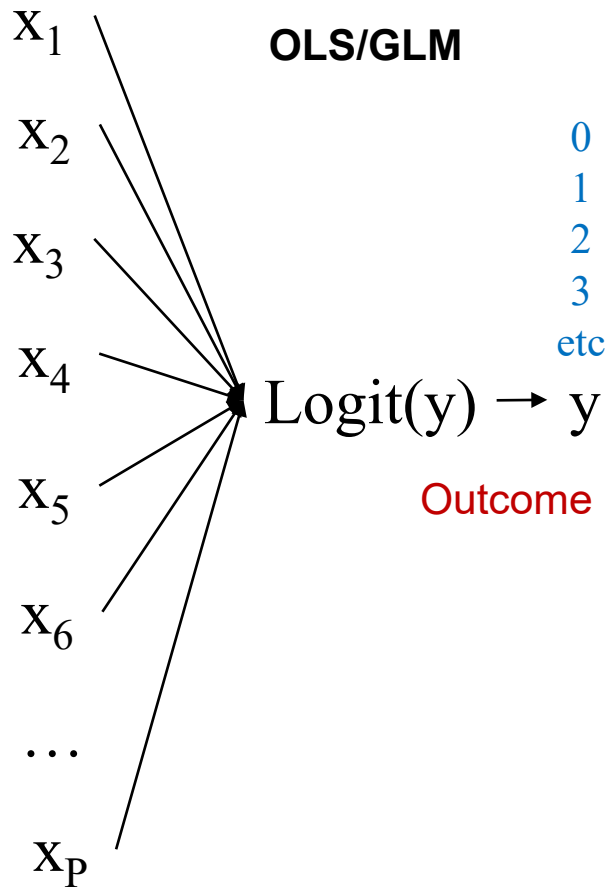


P Predictors
 N Data Points

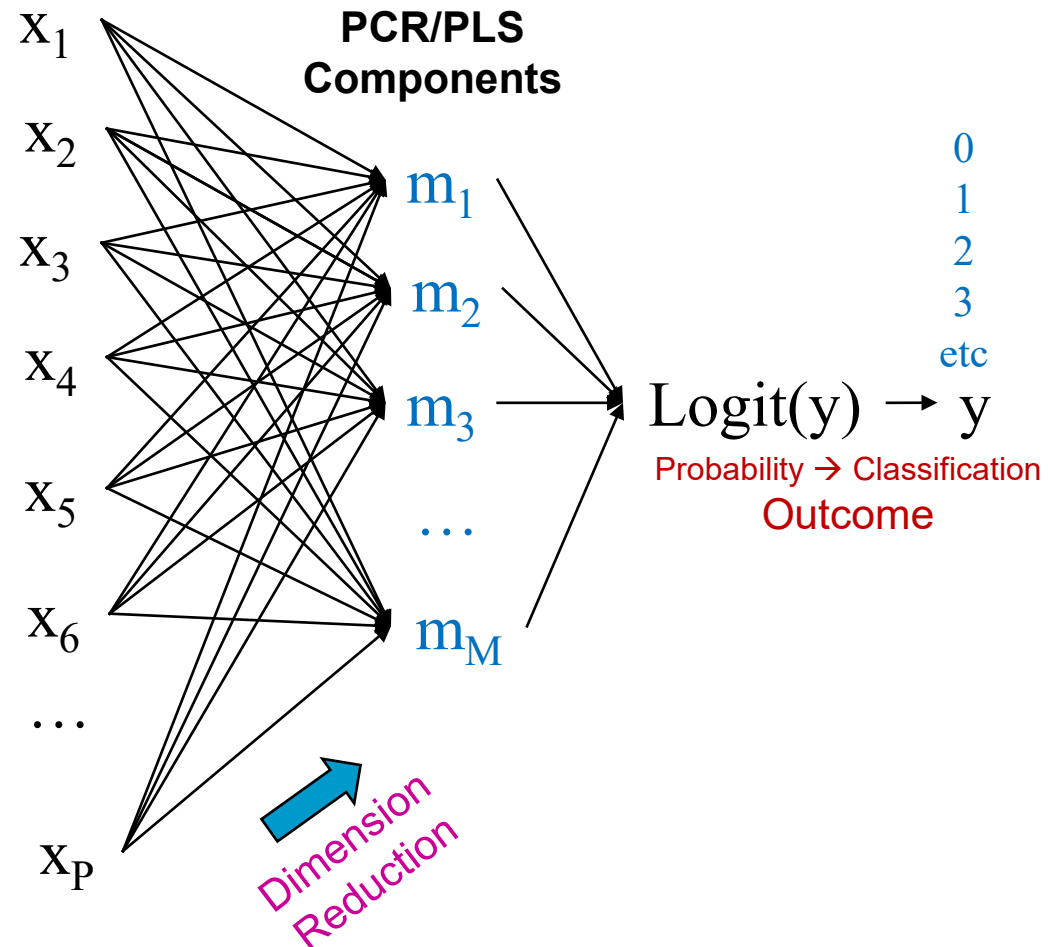


P Predictors
 N Data Points

Classification Regression & PCR/PLS



P Predictors
N Data Points



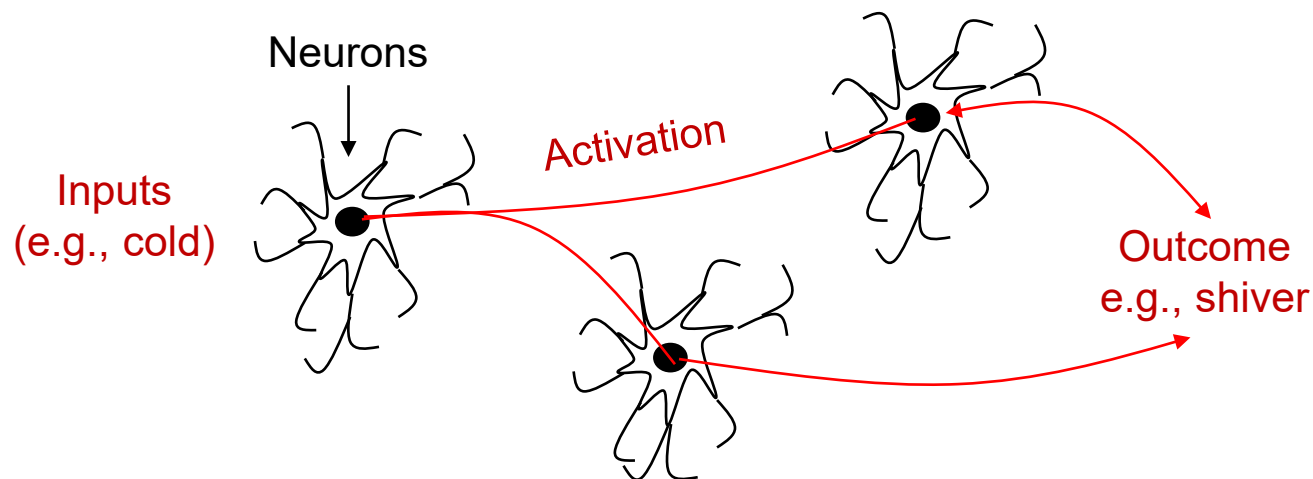
P Predictors
N Data Points

Biological Neural Networks

How the Brain Works

The human brain is a biological neural network composed of **billions** interconnected **neurons**, which are simple computational units that receive input signals from the environment (e.g., heat, cold, smell, pain, etc.) or other neurons and transmit that information to other neurons and so on, until it triggers a **behavioral** (e.g., put on a sweater) or **cognitive outcome** (e.g., fear, happiness).

Signals travel across complex paths and over **85+ billion of neurons** (cat 75M; cockroach 1M; fruit fly 100K), which enable complex learning



Artificial Neural Networks (ANNs or simply NNs)

How Neural Networks Work

An artificial **Neural Networks** (NN's) attempts to emulate how the brain works.

For example, what makes you stop or continue at a traffic light? In this case, a NN captures the image in front and processes every pixel color and feeds that into a predictive **binary classification model** → **stop** or **go** → the decision will be based on thousands of prior observations.

Classification NN's → Think of every pixel in an image as a predictor or input variable fed input into a **web of artificial neurons**, often grouped into **layers**, triggering an **activation** function that predicts a binary **outcome** → **stop** or **go**

Quantitative NN's → Works similarly to classification NN's, but the prediction is a quantitative value, not a classification.

Shallow and Deep Learning

In a NN, the inputs are fed into **neurons** organized in layers. These are not visible to the analyst, so they are referred to as **hidden layers**

A **hidden layer** can contain **one or more neurons**, and NNs can contain **1 or many hidden layers** of neurons.

NNs with **1 layer** are **simple NNs**, which are **shallow learning** models.

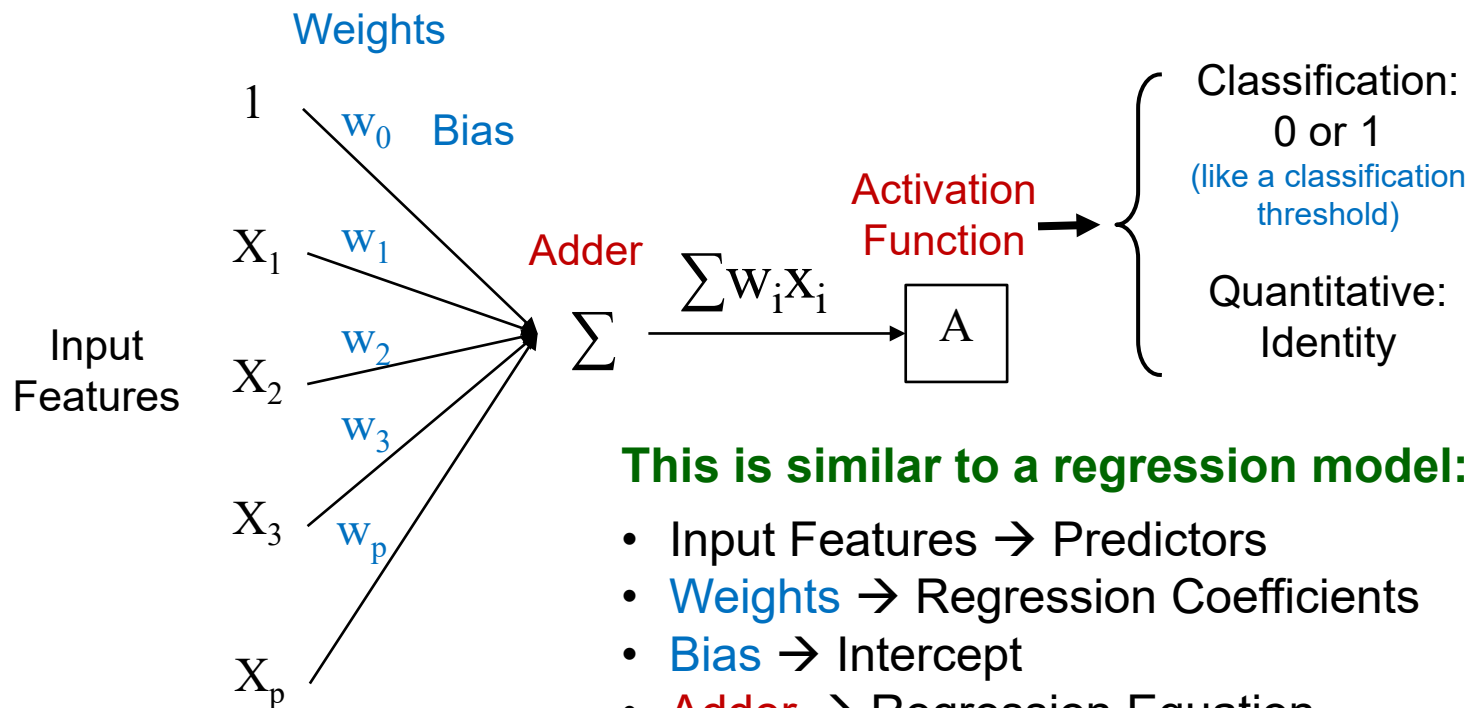
NNs with **more than one layer** are called **complex NNs**, which are **deep learning** models → the more layers, the deeper the learning.

Neurons also convert the outputs into final predictions using **activation functions**, which are transformations that let NNs model complex relationships between inputs and outcomes

In sum, a NN is similar to a **system of regressions** transforming and feeding outcomes as inputs to subsequent regressions, all the way to the final outcome. Each regression is assigned a **weight**, based on its **CV Test** accuracy. The **activation functions** enable the modeling of more complex relationships between neurons and outcomes.

The Perceptron

The **perceptron** is the simplest model of a **computational neuron** and constitutes the **basic component** of a neural network. All neural networks are composed of one or many perceptrons. A perceptron contains **input features**, **weights**, an **adder**, an **activation function** and an **output**.

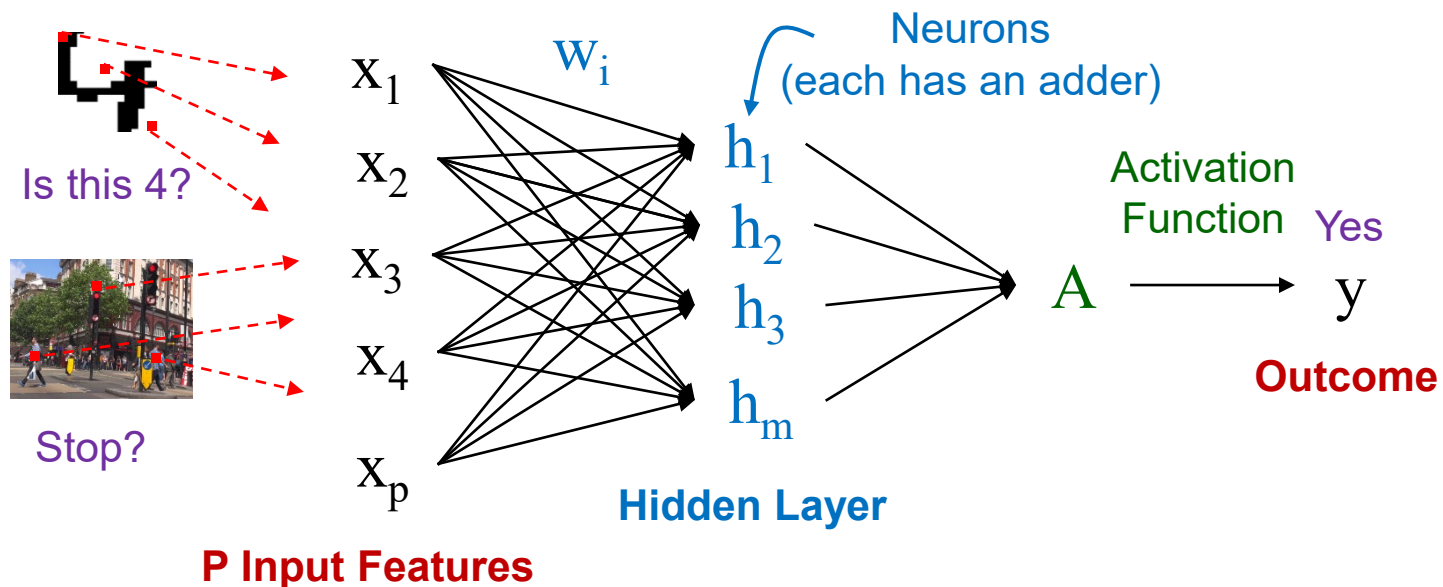


This is similar to a regression model:

- Input Features → Predictors
- **Weights** → Regression Coefficients
- **Bias** → Intercept
- **Adder** → Regression Equation
- **Activation Function** →
 - ✓ **Sigmoid** (reverse Logit), if classification
 - ✓ **Identity**, if quantitative

Simple Neural Network

It contains several input features and **multiple perceptrons** with **neurons** grouped in a **single hidden layer**. Each neuron has its own **adder** with its respective set of **weights**. It is similar to **PCR** and **PLSR**, except that the neurons are not principal components, but are estimated to maximize accuracy. It's a bundle of **regression equations**, each predicting a neuron outcome, which in turn predict the final **outcome**



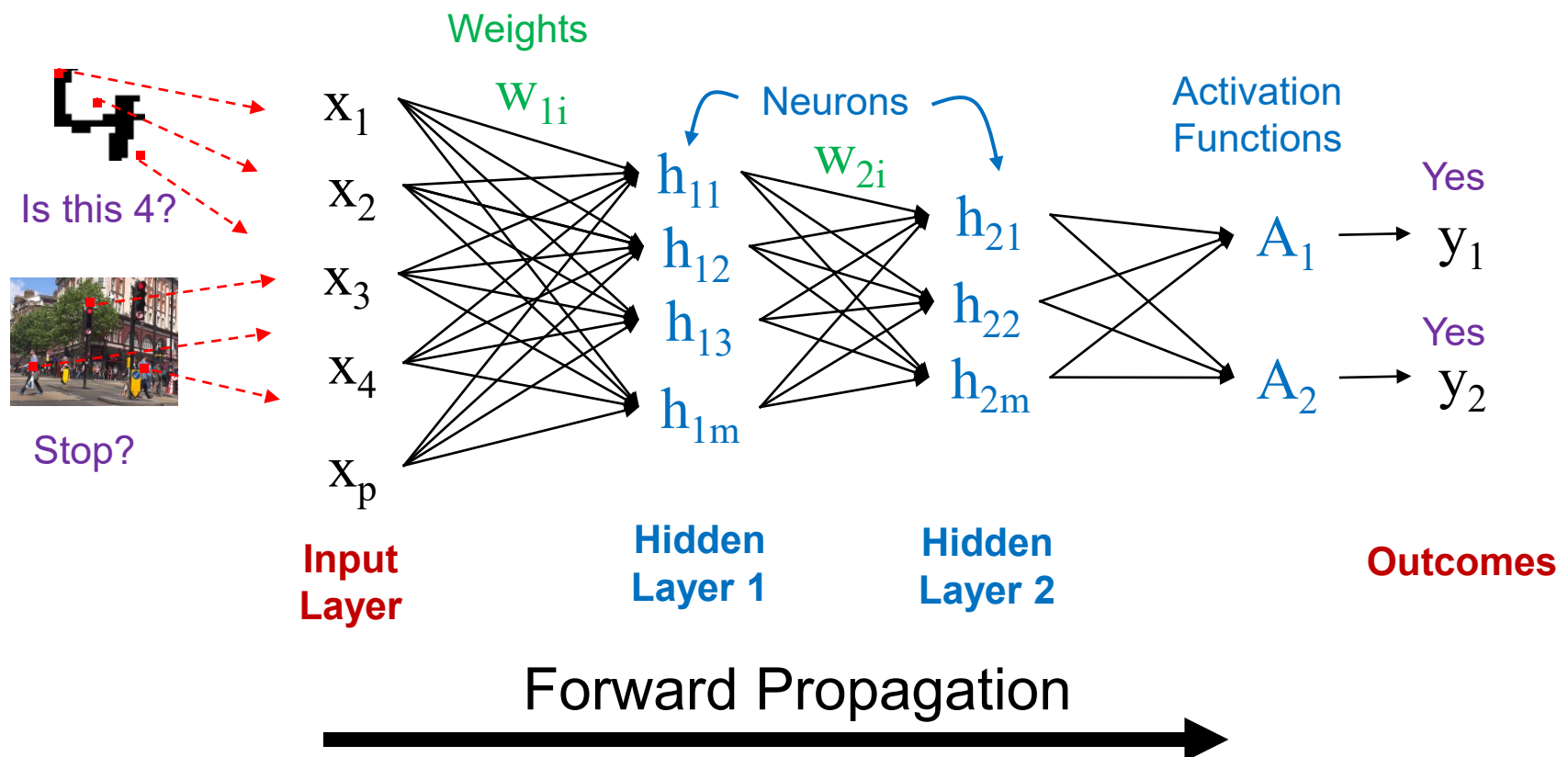
Feedforward Neural Networks

- In the NNs described so far, the **information flows in one direction**, from left to right.
- This is known as a **Feedforward NN**
- There are also **feedback**, **convolutional** or **recurrent NNs** with some information flowing back or even looping, but these are very complex and **difficult to train**
- **Feedforward NNs** start with a weighted sum of the input features, which are fed to a neuron in the first hidden layer → the same is done for all neurons in the layer, each with a different set of weights → these neurons then act as inputs for the neurons in the next hidden layer, and so on, until we reach the final outcome.
- This is called **forward propagation**.

Complex Neural Network (Deep Learning)

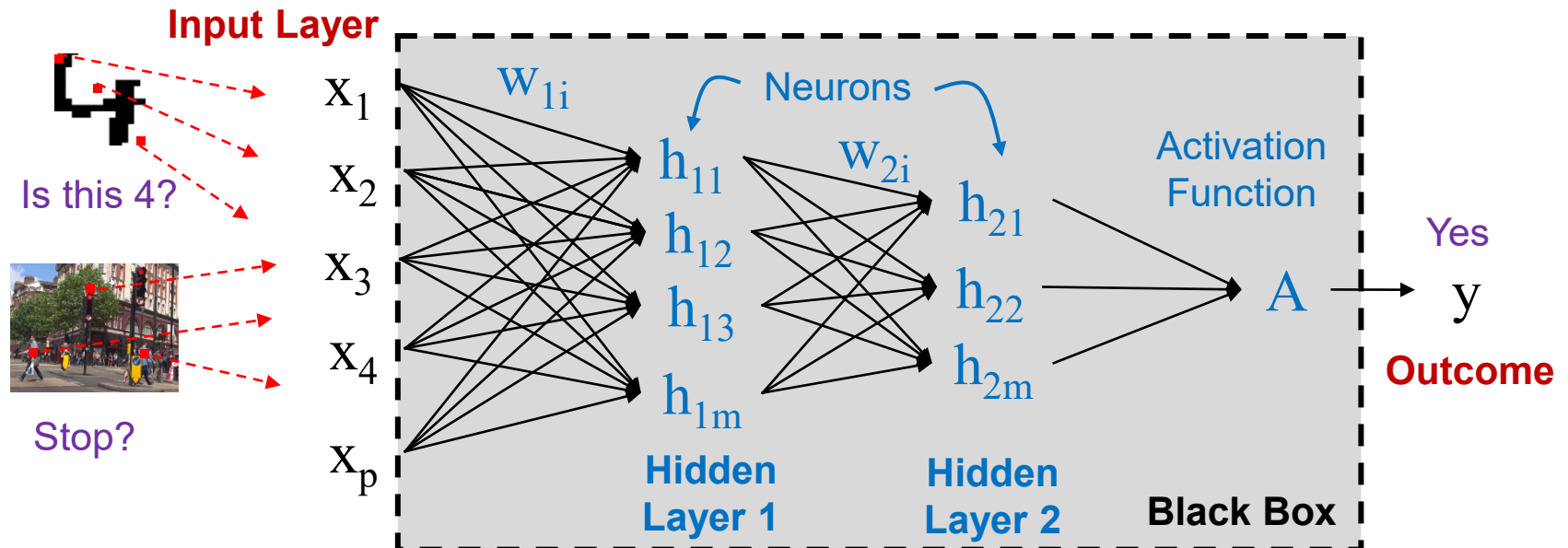
- **Complex NNs** contain **2 or more hidden layers** → the number of layers and neurons within each layer are **tuning parameters**
- The number of layers in a neural network and how neurons are organized is called **network topology**
- There are different types of **deep learning** models → a complex neural network is one of them → **more** hidden layers → **deeper** learning.
- More layers require much more **computational power** to train the model → it may or may not be more accurate than simple neural networks → and may not converge to a solution

Complex Feedforward Neural Network (FNN) (Illustration)

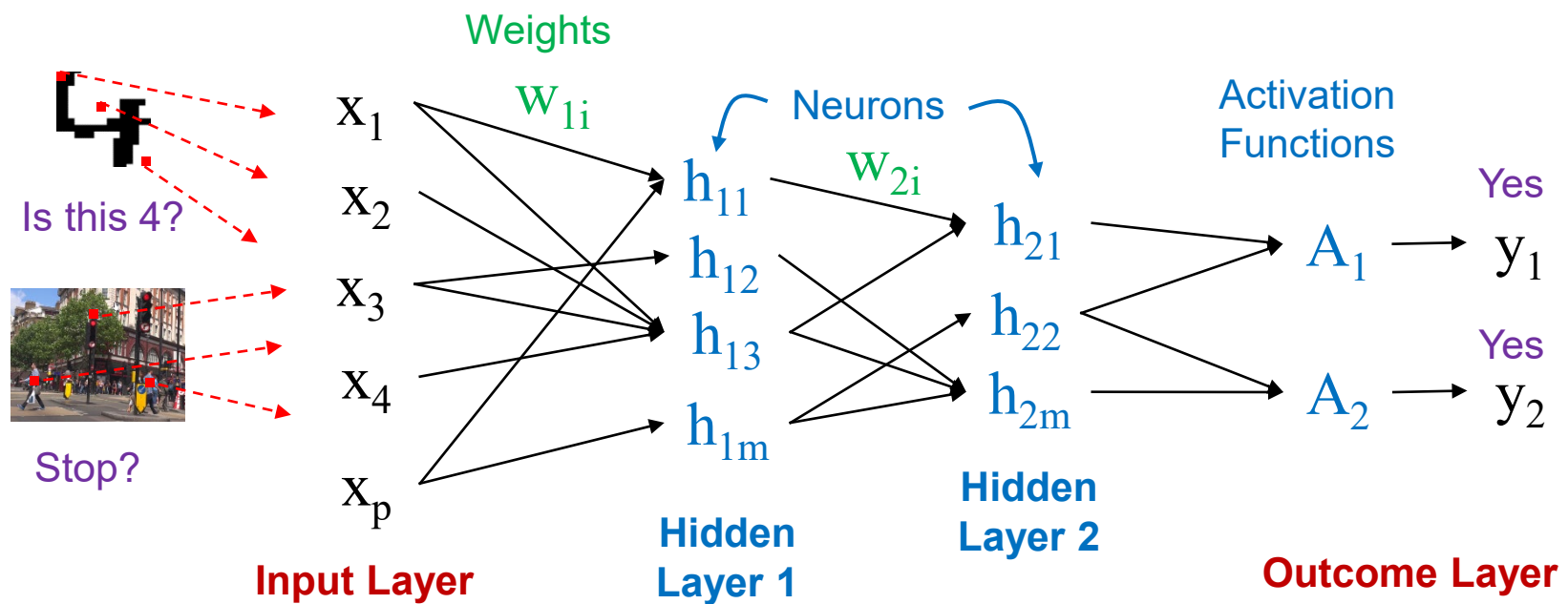


Black Box Models

- NNs and their math can be very **complex**
- **Initial arbitrary input weights** are selected to compute the **outcome variable** which is then **CV Tested** for accuracy.
- The process then **loops back** to adjust the input weights based on the CV Test results. This is repeated recursively in **many iterations**.
- **NNs** are **not** used for **interpretation**, but for **predictive accuracy**. They are **Black Box Models** because we don't need to see the internal math → The goal is to find input weights that **maximize CV Test accuracy**

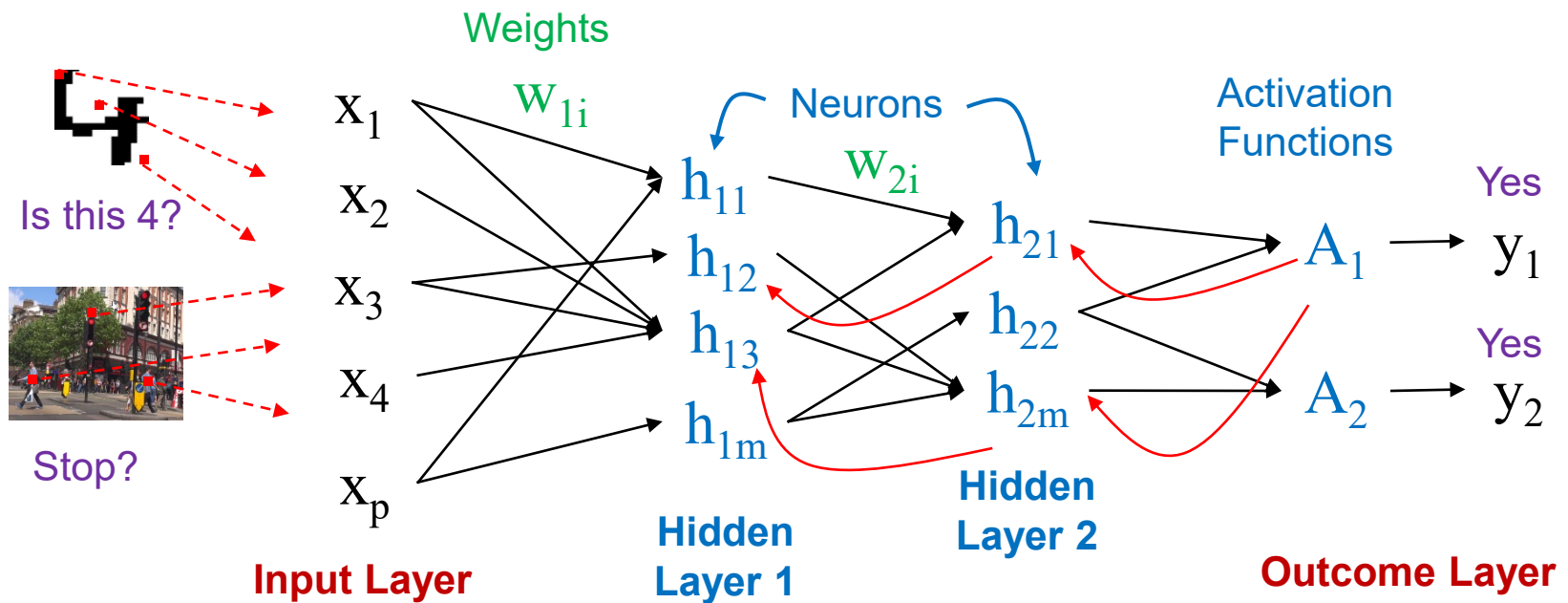


Convolutional Neural Network (CNN) (Illustration)



Not every input or node is connected to every node in the next layer

Recurrent Neural Network (RNN) (Illustration)



The prediction is not just forward, but can also be backwards

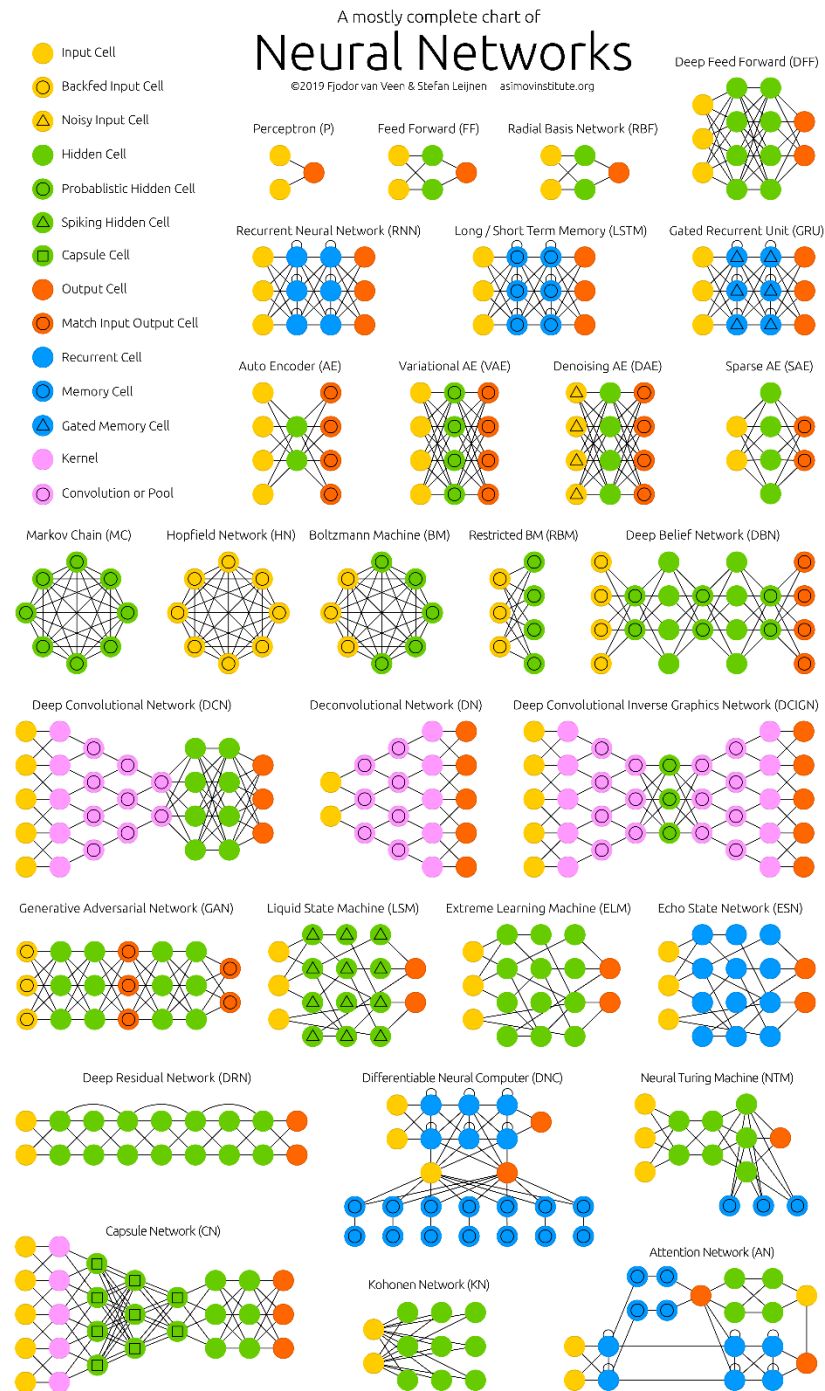
Neural Network Zoo

<https://www.asimovinstitute.org/neural-network-zoo/>

When to use which one?

Depends on the application, e.g.:

- Self-Driving Cars - Convolutional NN's (CNN's)
- Face Recognition - CNN's
- Text interpretation - Recurrent NN's (RNN's)
- Etc.



Training Neural Networks

- The **cost function** contains the **deviance** the model seeks to minimize
- For example, the cost function in **OLS** is the **CV Test MSE** and in **Logistic** regression is the **misclassification** rate
- Any **CV method** can be used → holdout sample, **KFCV**, **LOOCV**, etc.
- **Weights** for each neuron's adder are chosen randomly initially and the resulting **CV error** on the **outcome** variable is computed
- We then **back track** the **CV error** for each of the weights we use and **adjust** the **weights** that had larger errors
- We then use the adjusted weights to **recompute** the outcome and CV error, which should be lower
- We then **repeat** this process many times → each iteration called an **Epoch** → and continue until the CV error does not decrease any more
- When a minimum is found, the algorithm is said to have **converged**
- Complex NNs sometimes don't converge to a solution, but adjusting the parameters of the model often help the model converge

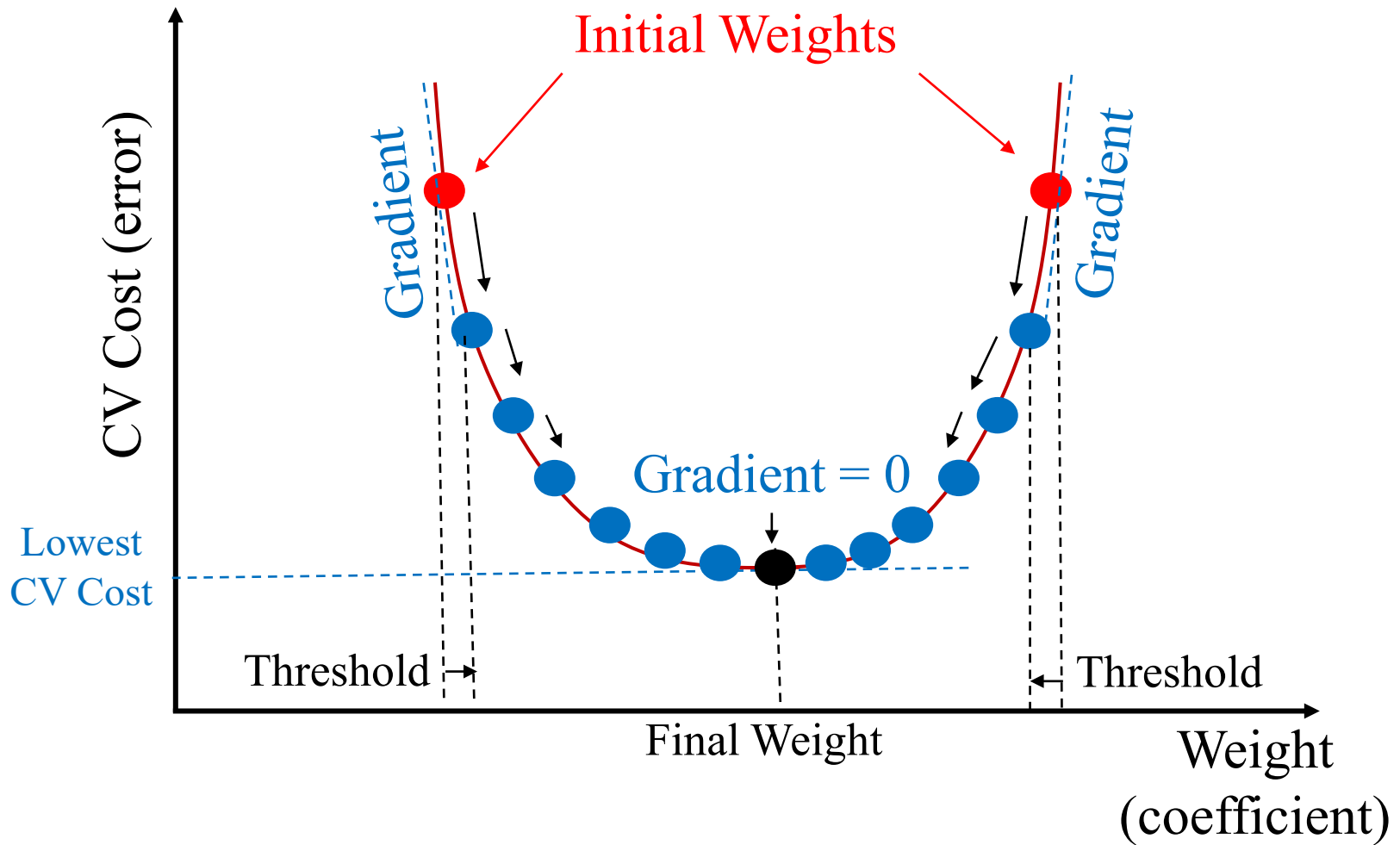
Back Propagation

- **Back propagation** is the most **common algorithm** to **train NNs**
- **Forward propagation** begins in the **input** layer and ends with the **output** layer, computing weights and hidden layer values in a forward fashion
- **Back propagation** is the process of computing the **CV cost (error)** of the **output** layer and propagating the resulting errors **backwards** and using this information to **update** all **weights**, based on where the largest errors were found, all the way back to the **input** layer
- This process allows us to find out where and how to **update** the **weights** to further reduce errors

Gradient Descent

- Is a popular method used to adjust **weights** (coefficients) in the **back propagation** process, to find the weights that **minimize the CV Cost** (error) in the output variable.
- If you plot the various values of a **predictor weight** (coefficient) against the **CV Cost** (error), the plot will often have a **U shape**.
- The **bottom of the curve** yields the value of the weight (coefficient) that **minimizes the CV Cost** (error).
- The idea is to start with **2 extreme values** of the weight (coefficient) – the end points of the U curve and to systematically increase or decrease these weights by a **fixed** proportion called the **“threshold”**.
- If the CV Cost decreases, we keep changing the weight by the threshold proportion on both sides of the curve, until the **CV Cost curve flattens** → i.e., the optimal value of the weight (coefficient).
- That is, we **“descend”** through the **“gradient”** of **CV Cost curve** until we get to the bottom, like rolling a ball down the hill.
- But do this **simultaneously** with **all** the **weights** (coefficients)

Gradient Descent Illustration



Pre-Trained Neural Networks

- The **accuracy** of a Neural Network and the resulting **predictor weights** are **heavily influenced** by the initial weights for the **first iteration**
- A popular method to expedite the gradient descent calculations for a Neural Network is to **start** with a **given set of weights** for each variable, rather than with a random set of weights (e.g., standardized regression coefficients)
- These weights can come from a **previously trained Neural Network** or some **other model**
- For example, you could start with an **OLS** or **Logistic** regression model and use the resulting coefficients as the starting weights for the Neural Network training
- These are called **Pre-Trained Neural Networks**
- Models trained this way are referred to as **Pre-Trained Models**.



Training Neural Nets w/neuralnet()

neuralnet() Function and Arguments Overview

Important Notes:

- # 1. **neuralnet()** requires quantitative predictors. If you have binary predictors, ensure they are numeric. If you have categorical variables, you need to transform them into the respective category numeric binary variables. See R script for more details.
- # 2. **neuralnet()** requires the data to be **normalized** or **standardized**, especially if quantities are large, or the model may not converge. Standardizing works many times, but experts recommend using a **0-1 min-max normalization**, i.e., $(x - \min(x)) / (\max(x) - \min(x))$. See R script.

Key neuralnet() Parameters (values indicated are the default)

- # **hidden = 1** – a vector containing the number of nodes in each layer, e.g., **hidden = 3** has 1 layer with 3 nodes; **hidden = c(4, 3)** has 2 layers, the first one with 4 nodes and the second one with 3
- # **threshold = 0.01** – is an approximation threshold used in gradient descent derivatives. The default value is 0.01 and the smallest value needed for neuralnet() to work is 0.01. If the model does not converge, increase the threshold progressively to 0.1, 0.2, etc. If the model does not converge still, increase it to 1, 10, 100, etc. Thresholds need to be ≥ 0.01 for neuralnet() to work properly.

Cont'd.





Training Neural Nets w/neuralnet()

neuralnet() Function and Arguments Overview (# Cont'd.)

- # **stepmax** = **10 ^ 5** – is the maximum number of steps for training a neural network. The model training will stop if the model has not converged after the stepmax. Increasing the value of stepmax will make it more likely that the model will converge, but it will take substantially longer.
- # **rep** = **1** – is the number of times you want the neuralnet training to run. Usually, 1 is sufficient, but if you want more randomness and are willing to wait, you can use more reps.
- # **algorithm** = **"rprop+"** – is the internal algorithm to calculate the neural network. Available methods include: "rprop+" (resilient back propagation), "rprop-", "backprop" (back propagation), "sag", "slr"
- # **err.fct** = **"sse"** – is the method used to calculate the CV test error. **"ce"** is cross-entropy, which can be used for classification models.
- # **act.fct** = **"logistic"** – the activation function, "logistic" and "tanh" are popular functions for classification models
- # **linear.output** = **T** – leave default as **T for quantitative** models or change to **F for classification** models, along with the **err.fct** and **act.fct** activation function





Example: Neural Networks

- `{library}` Topic
Search keywords
- Random Forest Trees
Multiple Linear Regression (for comparison)
Training NNs with a Quantitative Outcome
Training NNs with a Classification Outcome
Confusion Matrix